Time–Memory Trade-off in Computing the Minimum Hamming Distance of 32-Bit Words: Brute Force versus Sort-and-Search

Ahmed Saad Sweffi
Egypt-Japan University of Science and Technology (E-JUST), Alexandria, Egypt
ahmed.abdelgaber@ejust.edu.eg

***Abstract*—The task was to write a C function that returns the minimum Hamming distance between the 32-bit words of an array, keeping both the processing time and the memory size small. Since these goals conflict, I wrote two versions. Version 1 compares every pair of words and needs no extra memory. Version 2 sorts a copy of the array, searches for pairs at distance 0 and 1, and only then compares pairs, stopping at the smallest distance still possible. I ran both in C on a PC and on a Zynq-7020, as ARM Cortex-A9 software and as Vitis HLS cores, and they agree on every test. On 5,000 words, version 2 is 51 times faster on the PC, 37 times on the ARM and 7.8 times as an FPGA core, while version 1 needs no extra heap, 53 times less code and data on the ARM and 4.7 times fewer LUTs.**

***Index Terms*—Hamming distance, time–memory trade-off, sorting, binary search, popcount, Zynq-7000, Vitis HLS, FPGA**

I. Introduction

The task I worked on was the following:

> *"Write a C function that computes the minimum Hamming distance between the 32-bit words of an array, and optimize the processing time and the memory size required to compute the result as much as you can."*

The function has a fixed prototype:

```
int min_Hamming(unsigned int tab[], unsigned int tab_size);
```

The Hamming distance between two words is the number of bit positions where they differ, which is the number of 1-bits in their XOR [1]. The function returns the smallest such distance over all pairs of words in `tab`, a value from 0 to 32, or -1 when `tab` holds fewer than two words.

The last part of the task is the hard one, because time and memory pull in opposite directions: the fast way to find close words is to build extra data, such as a sorted copy of the array, while the smallest program compares the words where they are. A single version would end up somewhere in between, so I wrote two and measured both goals for each: version 1 (`min_hamming_v1.c`) aims at the smallest memory and version 2 (`min_hamming_v2.c`) at the shortest time.

I tested both versions on two platforms: in C on a PC, and on a Zynq-7020 SoC (XC7Z020-1), where each version runs as bare-metal software on the ARM Cortex-A9 and as a hardware core made with Vitis HLS [5] in the FPGA fabric.

Listing 1. min_hamming_v1.c (comments shortened)

```c
int min_Hamming(unsigned int tab[], unsigned int tab_size) {
    if (tab_size < 2)          // no pair, no answer
        return -1;
    int best = 32;             // words differ in at most 32 bits
    for (unsigned int i = 0; i < tab_size; i++)
        for (unsigned int j = i + 1; j < tab_size; j++) {
            unsigned int diff = tab[i] ^ tab[j]; // 1 = differ
            int distance = 0;
            for (int bit = 0; bit < 32; bit++)   // bit by bit
                if ((diff >> bit) & 1u)
                    distance++;
            if (distance < best)
                best = distance;
        }
    return best;
}
```

II. The Two Versions

A. Version 1: Brute Force (Memory Size)

Version 1 (Listing 1) is the direct method. For every pair of words it computes the XOR, counts the 1-bits one at a time with a shift-and-test loop, and keeps the smallest count. It always goes through all $N(N-1)/2$ pairs, so its time is $O(32N^2)$ whatever the input. On the other hand, it needs only a few registers (no `malloc`, no library call, no copy of the data), so its extra memory is $O(1)$.

B. Version 2: Sort and Search (Processing Time)

Version 2 (Listing 2) is based on a simple fact: the answer is usually small, and a small distance can be found without trying every pair. So it checks the possible answers from the smallest one up:

1) *Distance 0.* It sorts a copy of the array with `qsort`. Equal words end up next to each other, and one pass over the copy finds them. Cost: $O(N \log N)$.
2) *Distance 1.* For every word, it flips each of the 32 bits in turn and looks for the result in the sorted copy with a binary search [2]. A hit means distance 1. I wrote the search by hand, because `bsearch` would call the comparison function through a pointer at every step. Cost: $O(32N \log N)$.
3) *Distance 2 or more.* It compares all pairs like version 1, but counts the bits with one instruction (GCC turns `__builtin_popcount` into `POPCNT`), and it stops at the first pair at distance 2, since nothing smaller is left. The

TABLE I
THE TWO VERSIONS AT A GLANCE (N WORDS)

	Version 1	**Version 2**
Idea	XOR every pair, count the bits one by one	sort a copy, search the one-bit neighbours, then scan the pairs with an early exit
Time	$O(32N^2)$, always all pairs	$O(N \log N)$ if the answer is 0, $O(32N \log N)$ if it is 1, else $O(N^2)$ at worst
Extra memory	$O(1)$, no allocation	$O(N)$: $8N$ B on the PC, $4N$ B on the ARM
Cost per pair	267 instructions (PC), 37 (AVX2 build), about 201 (ARM)	8 instructions (PC), about 21 (ARM, which has no popcount instruction)

worst case is still $O(N^2)$, but a pair now costs about 8 instructions instead of 267.

The price is memory. The sorted copy takes $4N$ bytes (I do not sort `tab[]` itself, since the caller does not expect it to change), and glibc's `qsort` adds a $4N$-byte merge buffer, so the peak on the PC is $8N$ bytes; newlib's `qsort` on the ARM sorts in place, so there it is $4N$. If `malloc` fails, version 2 skips steps 1 and 2 and still returns the right answer, only more slowly. Table I compares the two versions.

C. The FPGA Cores

For the FPGA, I rewrote both algorithms in C++ for Vitis HLS with the same interface. Each core takes up to 8,192 words (what fits on chip) and returns -2 for more. The version 1 core copies the words from DDR into 16 block-RAM banks, then checks 16 pairs per clock, each with an XOR and an adder tree that counts the bits, and never stops early. The version 2 core keeps the three steps with parts that suit hardware: a 4-pass radix sort [2] instead of `qsort`, a 13-level search tree, built twice, that answers 4 look-ups per clock, and a pair scan of 64 pairs per clock with an early exit. To feed them, it keeps six copies of the input on chip.

III. TESTING AND BENCHMARKS

A. Benchmark and Tests

The same `main.c` drives both versions. It runs 6 tests with answers worked out by hand (a general case, a duplicate, one bit apart, two bits apart, the largest distance and a single word: 4, 0, 1, 2, 32 and -1). Then it times one call on 5,000 words from the MT19937 generator [3] with seed 42. Each build ran 15 times, and I report the median.

The smallest distance in this array is 2. Three pairs reach it, and the first is words 263 and 3,180 (Table II). Version 1 checks all 12,497,500 pairs. Version 2 finds no duplicate in step 1 and no one-bit neighbour in step 2 (160,000 binary searches), then stops in step 3 after 1,283,201 pairs, about 10 % of them.

A sweep program (`bench.c`) times both versions on 5 sizes (1,000 to 20,000 words), 7 kinds of input and both builds, 70 measurements in all, on one pinned core (median of 3 to 9 calls). Besides main.c's array, the inputs use code words of the extended Hamming code $[32, 26, 4]$ [1], where any two words differ in at least 4 bits, with one planted pair at distance 0, 1 or 2, or with none; then the answer is 4 and version 2 cannot stop early.

Both versions return the same answer on every test and on 20,005 random arrays of 0 to 2,500 words. I measured the heap with Valgrind Massif [4], the stack with a painted-stack program (`stack2.c`) that runs each call on a fresh stack filled with a pattern and counts the bytes that changed, and the instructions inside the call with Callgrind.

Listing 2. min_hamming_v2.c (comments shortened)

```c
#include <stdlib.h>   // malloc, free, qsort
#include <string.h>   // memcpy
// one POPCNT instruction with gcc -mpopcnt (the file also has
// a portable bit count for other compilers)
#define POPCOUNT32(x) __builtin_popcount(x)
static int compare_words(const void* pa, const void* pb) {
    unsigned int a = *(const unsigned int*)pa;
    unsigned int b = *(const unsigned int*)pb;
    return (a > b) - (a < b);    // a - b could overflow
}
int min_Hamming(unsigned int tab[], unsigned int tab_size) {
    if (tab_size < 2)
        return -1;
    int lowest = 0;              // smallest answer still possible
    unsigned int* sorted =
        malloc((size_t)tab_size * sizeof(unsigned int));
    if (sorted != NULL) {        // no memory: skip steps 1 and 2
        memcpy(sorted, tab, (size_t)tab_size * sizeof(unsigned int));
        qsort(sorted, tab_size, sizeof(unsigned int), compare_words);
        // step 1: equal neighbours -> distance 0
        for (unsigned int i = 0; i + 1 < tab_size; i++)
            if (sorted[i] == sorted[i + 1]) {
                free(sorted);
                return 0;
            }
        // step 2: binary search for the 32 one-bit neighbours
        for (unsigned int i = 0; i < tab_size; i++)
            for (int bit = 0; bit < 32; bit++) {
                unsigned int neighbour = sorted[i] ^ (1u << bit);
                unsigned int lo = 0, hi = tab_size;
                while (lo < hi) {
                    unsigned int mid = lo + (hi - lo) / 2;
                    if (sorted[mid] < neighbour) lo = mid + 1;
                    else                         hi = mid;
                }
                if (lo < tab_size && sorted[lo] == neighbour) {
                    free(sorted);
                    return 1;
                }
            }
        free(sorted);
        lowest = 2;              // no pair at distance 0 or 1
    }
    // step 3: all pairs, one-instruction bit count, early exit
    int best = 32;
    for (unsigned int i = 0; i < tab_size; i++)
        for (unsigned int j = i + 1; j < tab_size; j++) {
            int distance = POPCOUNT32(tab[i] ^ tab[j]);
            if (distance < best) {
                best = distance;
                if (best == lowest)  // nothing smaller is possible
                    return best;
            }
        }
    return best;
}
```

TABLE II
CLOSEST PAIR OF THE BENCHMARK ARRAY: WORDS 263 AND 3,180 DIFFER ONLY IN BITS 9 AND 4

	Hex	**Binary, bit 31 ... bit 0**
word 263	B1437CE0	1011 0001 0100 0011 0111 11**0**0 111**0** 0000
word 3,180	B1437EF0	1011 0001 0100 0011 0111 11**1**0 111**1** 0000
XOR	00000210	0000 0000 0000 0000 0000 00**1**0 000**1** 0000

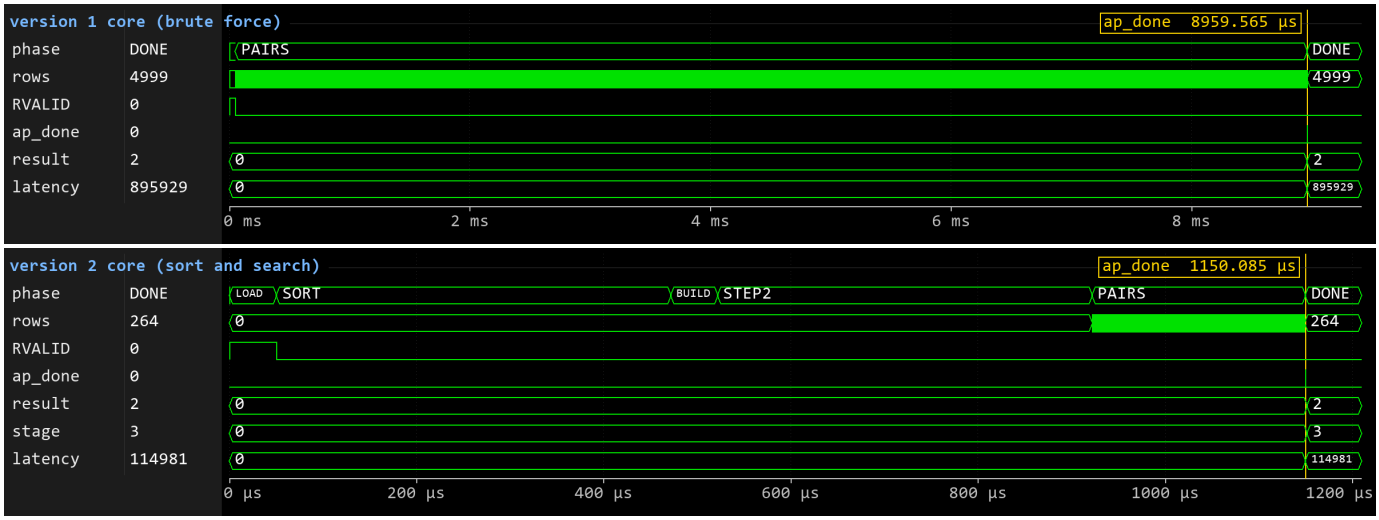


Fig. 1. RTL simulation of both HLS cores on main.c's 5,000 words at 100 MHz (Icarus Verilog, drawn from the VCD files). `phase`: the running step; `rows`: rows (version 1) or blocks (version 2) of pairs started; `RVALID`: words arriving from DDR; `latency`: cycles from `ap_start` to `ap_done`. Next to the names are the final values; the cursor marks `ap_done`.

B. RTL Simulation

I also simulated the Verilog that Vitis HLS generated for each core with Icarus Verilog [8], using a testbench (`tb_min_hamming.v`) that acts like the ARM program: it writes the array address, the size and `ap_start` over AXI4-Lite [7], serves the core's AXI4 read bursts from a memory model holding the same 5,000 words, and polls `ap_done`. Fig. 1 shows both runs. The version 1 core copies the words in (50 μs), works through 4,999 rows of pairs and finishes after 895,929 cycles (8.96 ms at 100 MHz). The version 2 core loads (50 μs), sorts (421 μs), builds its tree (50 μs) and runs step 2 (400 μs); step 3 then finds a distance-2 pair after 264 blocks, and the core finishes after 114,981 cycles (1.150 ms), as on the board. Both return 2. On the board, version 1 took 9.008 ms because its timer also covers the cache flush and the register accesses.

IV. Platforms and FPGA Deployment

PC. An Intel Core i5-1135G7 (Tiger Lake, up to 4.2 GHz) running Ubuntu 24.04 with GCC 13.3, glibc 2.39 and Valgrind 3.22; calls are timed with `clock_gettime`. I used two builds: scalar (`gcc -O2 -mpopcnt -std=c11`) and AVX2 (`gcc -O3 -march=native -std=c11`). In the AVX2 build, GCC vectorizes the inner loop of version 1, while the early-exit loops of version 2 stay scalar.

Boards. Version 1 ran on a Zybo Z7-20 and version 2 on a ZC702, both with the XC7Z020-1 chip and the same clocks: 666.67 MHz for the ARM, 533 MHz for the DDR3 and 100 MHz for the FPGA logic. The ARM code is bare metal (standalone BSP, `arm-none-eabi-gcc -O2`, newlib), timed with the Cortex-A9 global timer; the tools were Vitis HLS, Vivado and Vitis 2021.1.

Block design. Each core is an IP block next to the Zynq processing system [6] (Fig. 2). The ARM drives the core's AXI4-Lite registers at `0x43C0_0000` through `M_AXI_GP0` and an AXI Interconnect: start and done at 0x00, the result at 0x10, the array address at 0x18 and the size at 0x20. The core reads the words itself: its AXI4 master, `m_axi_gmem`, goes through a second interconnect into `S_AXI_HP0` and on to DDR, in bursts. The program flushes the array from the data cache, writes the registers, starts the core and polls `ap_done`. Since version 2 calls `malloc`, its board build reserves a 1 MB heap (`-Wl,--defsym=_HEAP_SIZE=0x100000`). With the BSP default of 8 KB, `malloc` would fail near $N = 2{,}048$ and version 2 would lose steps 1 and 2 (but still answer correctly).

Listing 3. UART output of the version 2 board program (excerpt)

```
-- Benchmark: N = 5000 random words (MT19937, seed 42) --
  ARM version 1 : distance 2, 2699.550 ms
  ARM version 2 : distance 2, 72.437 ms  (fastest of 5)
  PL core       : distance 2, 1.150 ms .. 1.151 ms  (10 runs, decided in step 3)
-- Random regression: 300 arrays, PL core vs ARM version 2 --
  0 mismatches; decided by step 1: 52, step 2: 69, step 3: 179
ALL TESTS PASSED
```

Board tests. Version 1 passed 7 tests: main.c's six on both the core and the ARM, and an array of 8,193 words that the core rejects with -2. Version 2 passed 8 tests (the same six, an empty array and 8,193 words) and 300 random arrays checked against the ARM version with no mismatch (Listing 3). Both ARM times in Section V come from the version 2 program, which contains both versions; version 1's own program measured 2,737.1 ms.

V. Results and Comparison

Table III lists all the results. The estimates come from Vivado's power report and the HLS cycle model and are marked as such; the rest were measured.

Processing time: version 2 wins.

- It is faster everywhere: 51× on the PC, 5.8× with AVX2, 37× on the ARM and 7.8× on the FPGA, with 101× fewer instructions.

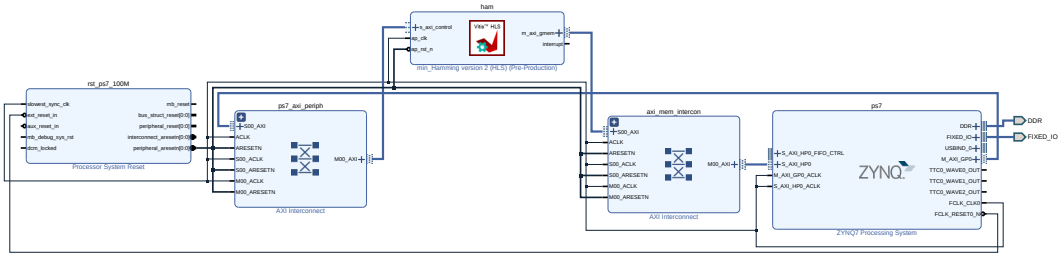


Fig. 2. Vivado block design of the version 2 system on the ZC702. The version 1 system on the Zybo Z7-20 has the same blocks and connections.

TABLE III
SCORECARD (ONE CALL, 5,000 WORDS, UNLESS NOTED)

Measure	Version 1	Version 2	Better	By
Processing time				
PC, scalar build	303.3 ms	5.90 ms	V2	51×
PC, AVX2 build	34.9 ms	5.98 ms	V2	5.8×
ARM Cortex-A9	2,699.6 ms	72.4 ms	V2	37×
FPGA core, 100 MHz	9.008 ms	1.150 ms	V2	7.8×
PC sweep, 70 cases	–	70 wins	V2	1.34–3,222×
No pair under 4 bits, PC	303.5 ms	10.8 ms	V2	28×
PC, duplicate, $N = 20{,}000$	5,030 ms	1.67 ms	V2	3,007×
Instructions, PC scalar	3,336.9 M	33.0 M	V2	101×
Memory size				
Extra heap, PC	0	$8N$ B	V1	–
Stack during the call, PC	<128 B	2,712 B	V1	>21×
Code, PC	222 B	606 B + libc	V1	2.7×
Extra heap, ARM	0	$4N$ B	V1	–
Code and data, ARM	128 B	6,810 B	V1	53×
FPGA LUTs	2,557	11,898	V1	4.7×
FPGA flip-flops	3,328	8,314	V1	2.5×
FPGA block RAM tiles	12	52.5	V1	4.4×
On-chip copies of the input	32 KB	192 KB	V1	6.0×
Time and size together (FPGA cores, estimates)				
Dynamic power	0.090 W	0.331 W	V1	3.7×
Energy per call	0.81 mJ	0.38 mJ	V2	2.1×
Same, no pair under 4 bits	0.81 mJ	1.13 mJ	V1	1.4×
Correctness, all tests	pass	pass	tie	–

- The gap follows the cost of one pair. AVX2 brings version 1 down to 37 instructions, and on the ARM, which has no popcount instruction, version 2 calls libgcc's `__popcountsi2` and needs about 21 against about 201.
- It wins all 70 sweep cases (1.34× to 3,222×), and even with no pair closer than 4 bits, where it cannot stop early, it stays 28× (scalar) and 2.9× (AVX2) faster.

Memory size: version 1 wins.

- PC: no extra heap against $8N$ bytes, under 128 B of stack against 2,712 B, and 222 B of code against 606 B plus `qsort`, `malloc` and `memcpy`.
- ARM: no extra heap against $4N$ bytes, and 53× less code and data with the library code counted (128 against 6,810 B).
- FPGA: 4.7× fewer LUTs, 2.5× fewer flip-flops and 4.4× less block RAM. The whole design uses 6.2 % of the LUTs instead of 23.8 %, with more slack at 100 MHz (+2.952 against +0.672 ns).

Time and size together. Version 2's core draws more power (0.331 against 0.090 W, estimate) but finishes 7.8 times sooner, so it needs 2.1 times less energy per call (0.38 against 0.81 mJ) and less LUT-time (13,683 against 23,033 LUT·ms). With no pair closer than 4 bits this reverses (1.13 mJ and 40,493 LUT·ms, estimate), and version 1 is the more efficient circuit, though 2.6 times slower.

Correctness is a tie: the same answers on every test, on the boards and on 20,005 random arrays.

VI. CONCLUSION

The two versions sit at the two ends of the time–memory trade-off. Version 2 wins every time measure on every platform, and its lead grows with N, since its sort and neighbour search grow as $N \log N$ while the pair scan that it usually cuts short grows as N^2; its memory cost stays a fixed multiple of the input. Version 1 wins every memory measure: no heap, a tiny stack, 128 B of code on the ARM and the smallest FPGA core.

I would use version 2 when time matters and the target can spare $O(N)$ memory: a PC or application processor, the Zynq ARM with a raised heap, or an FPGA with block RAM to spare. Version 1 fits targets with no heap or a few kilobytes of RAM, where code size counts, and FPGA designs where area or energy matter more than latency and close pairs are rare. Version 2's memory could also come down (not built or measured): sorting `tab[]` in place, if the caller allows it, removes the extra heap, and an in-place sort such as heapsort removes glibc's $4N$-byte buffer and most of the stack.

My answer to Problem 1 is version 2. Version 1 remains the right choice when memory, not time, is the limit.

REFERENCES

[1] R. W. Hamming, "Error detecting and error correcting codes," *Bell Syst. Tech. J.*, vol. 29, no. 2, pp. 147–160, Apr. 1950.

[2] D. E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd ed. Reading, MA, USA: Addison-Wesley, 1998.

[3] M. Matsumoto and T. Nishimura, "Mersenne twister: A 623-dimensionally equidistributed uniform pseudo-random number generator," *ACM Trans. Model. Comput. Simul.*, vol. 8, no. 1, pp. 3–30, Jan. 1998.

[4] N. Nethercote and J. Seward, "Valgrind: A framework for heavyweight dynamic binary instrumentation," in *Proc. ACM SIGPLAN Conf. Program. Lang. Design Implement. (PLDI)*, 2007, pp. 89–100.

[5] Xilinx, *Vitis High-Level Synthesis User Guide*, UG1399, 2021.

[6] Xilinx, *Zynq-7000 SoC Technical Reference Manual*, UG585.

[7] Arm, *AMBA AXI and ACE Protocol Specification*, IHI 0022.

[8] S. Williams, *Icarus Verilog*. [Online]. Available: https://github.com/steveicarus/iverilog